

Homework — 2.1.3 Thinking Procedurally — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [3] Any three plausible sub-procedures (1 each): browse/select film and showing; check seat availability; allow the user to choose seats; take/validate payment; issue/email the ticket; update the seat-availability records; create the booking record. Must be plausible sub-tasks of the booking scenario — generic answers ("do the input", "do the processing") earn nothing.
2. [2] 1 mark: `issueTicket` depends on the payment having succeeded. 1 mark: it uses the **output** of `takePayment` (confirmation that payment was successful); a valid ticket cannot be issued until payment is confirmed, so the order is fixed/sequential.
3. (a) [2] 1 mark for a valid fixed pair, e.g. "fill with water" before "heat water" (or "lock door" before "fill", "wash" before "drain", "drain" before "spin"). 1 mark: the dependency — e.g. water cannot be heated before it has been added; the later step needs the result/state produced by the earlier one.

(b) [1] The display update does not use the output of the wash step (and the wash does not use the display's output), so there is **no dependency** between them — the two tasks are independent and their relative order is not fixed. (*The mark is for identifying the lack of dependency.*)
4. [2] Any two (1 each): the code is defined in a **single place** and can be called for every question (reuse); a change or fix is made **once** and takes effect everywhere, avoiding inconsistent duplicated code; the sub-procedure can be tested/debugged separately; the program is shorter/easier to read.
5. [2] 1 mark: decomposition splits the game into smaller, self-contained sub-problems/modules (e.g. world rendering, combat, save system, AI). 1 mark: each module can be assigned to a **different programmer** to develop and test at the same time, then integrated — making the large task manageable.
6. [2] 1 mark: sensible named sub-procedures in a workable order, e.g. `receiveApplication` → `validateDetails` → `checkEligibility` → `takePayment` → `issuePermit`. 1 mark: a fixed-order pair with the dependency justified via the earlier step's output, e.g. `issuePermit` must follow `checkEligibility` because it uses the eligibility decision as its input — a permit cannot be issued before eligibility is confirmed. Must be applied to the scenario with named sub-procedures.

Part B (6 marks)

7. [2] 1 mark: a precondition is a condition that must be **true before** a routine runs for the routine to work correctly. 1 mark: for binary search, the list must already be **sorted** (accept: the list is not empty).
8. [2] 1 mark each, in the correct relative order, for any two of: **lexical analysis** (tokenising) → **syntax analysis** (parsing) → **semantic analysis**. (Code generation and optimisation follow.)
9. [2] 1 mark + 1 mark for one developed advantage: Agile is **iterative**, delivering working software in increments and accommodating changing requirements [1]; the customer sees progress early and

changes are cheaper to make, whereas Waterfall fixes requirements up front so late changes are costly [1].

Total: 14 + 6 = 20 marks